1. Abstract

This report presents a comprehensive technical overview of **ShechBot**, an Arduino UNO-based smart automated plant watering system designed for precision irrigation. The system integrates a DS3231 real-time clock (RTC) module calibrated to Bangladesh Standard Time, an analog soil moisture sensor, an IRFZ44N MOSFET power driver, and a 20×4 LCD display with a 4×4 matrix keypad to deliver a fully standalone, menu-driven irrigation controller powered by a 7.4 V Li-ion battery pack.

The device operates across two independent irrigation modes. In **Time-Based Mode**, the DS3231 RTC triggers a watering event at a user-configured hour and minute, checking soil moisture first to avoid unnecessary watering. In **Moisture-Based Mode**, the analog soil sensor is polled continuously and the pump activates automatically whenever moisture falls below a user-defined threshold, with a 5-minute cooldown to prevent flooding. Both modes are governed by a finite-state machine (FSM) firmware architecture written in C++ for the Arduino platform.

All user-configurable parameters — target moisture percentage, scheduled watering time, and pump run duration — are stored persistently in the microcontroller's internal EEPROM using an XOR checksum validation scheme, ensuring that settings survive power outages and battery replacements without requiring re-programming. A resistive voltage-divider circuit connected to an analog input pin continuously monitors the battery pack voltage and maps it to a 0–100 % charge estimate displayed live on the LCD screen.

Experimental validation conducted over a 30-day field trial confirmed reliable watering event execution (96.7 % scheduled success rate), accurate RTC timekeeping (< 3 seconds drift per month), and stable sensor readings throughout. The project demonstrates that a capable, low-cost smart-irrigation solution can be realised entirely from commodity embedded hardware without cloud dependency.

2. Introduction

Water scarcity and inefficient irrigation practices are among the most pressing challenges in modern small-scale agriculture and domestic plant care. Overwatering causes root rot and fungal disease, while underwatering leads to wilting and crop loss. Manually monitoring soil moisture and adjusting watering schedules is labour-intensive and inconsistent, especially in off-grid environments where internet connectivity and smartphone access cannot be relied upon.

ShechBot addresses this challenge by providing a fully autonomous, battery-powered irrigation controller built entirely from commodity Arduino hardware. Unlike cloud-based smart irrigation systems that require Wi-Fi, a subscription service, or a mobile application, ShechBot operates as a completely standalone device. All configuration is performed locally through the onboard LCD menu and 4×4 keypad, making it accessible to users with no technical background and deployable in remote field environments.

The system was designed around four engineering principles: **reliability** (EEPROM persistence, flyback protection, PWM ramp-up), **usability** (interactive LCD menu, screen-saver), **efficiency** (moisture-check before watering, 5-minute cooldown, one-trigger-per-day guard), and **maintainability** (modular FSM firmware, checksum-validated settings). These principles guided every hardware and firmware decision in this report.

Project Objectives

- Design a dual-mode system supporting both scheduled and moisture-triggered watering.
- Implement persistent EEPROM-based settings storage with XOR checksum validation.
- Develop a fully interactive LCD/keypad interface for in-field parameter adjustment.
- Integrate real-time battery voltage monitoring to support mobile deployment.
- Validate system reliability through controlled testing over a 30-day period.

3. System Design and Methodology

3.1 Hardware Architecture

ShechBot's hardware design prioritises modularity, low-power operation, and field replaceability. Table 1 lists all components with their specifications and roles. Figure 1 shows the full combinational circuit and product diagram.

Component	Model / Specification	Function in System
Microcontroller	Arduino UNO (ATmega328P)	Central processor — executes all system logic
RTC Module	DS3231 (±2 ppm TCXO)	Maintains Bangladesh Standard Time for scheduling
Power Transistor	MOSFET IRFZ44N	Amplifies gate signal to drive the water pump
Protection Diode	1N4007	Flyback diode — suppresses reverse EMF at cutoff
Soil Sensor	Analog capacitive sensor	Reads live soil moisture (mapped 0–100 %)
Resistors	1kΩ / 5kΩ / 10kΩ	Voltage divider for battery voltage measurement
Display	20×4 I2C LCD (0x27)	Shows menus, live readings, and system status
Keypad	4×4 Matrix Keypad	User input for navigation and configuration
Battery	7.4 V Li-ion pack	Portable rechargeable power supply
Charging Module	2S Li-ion charger board	Safely recharges the 7.4 V battery in the field

Table 1 — Full hardware and equipment list.

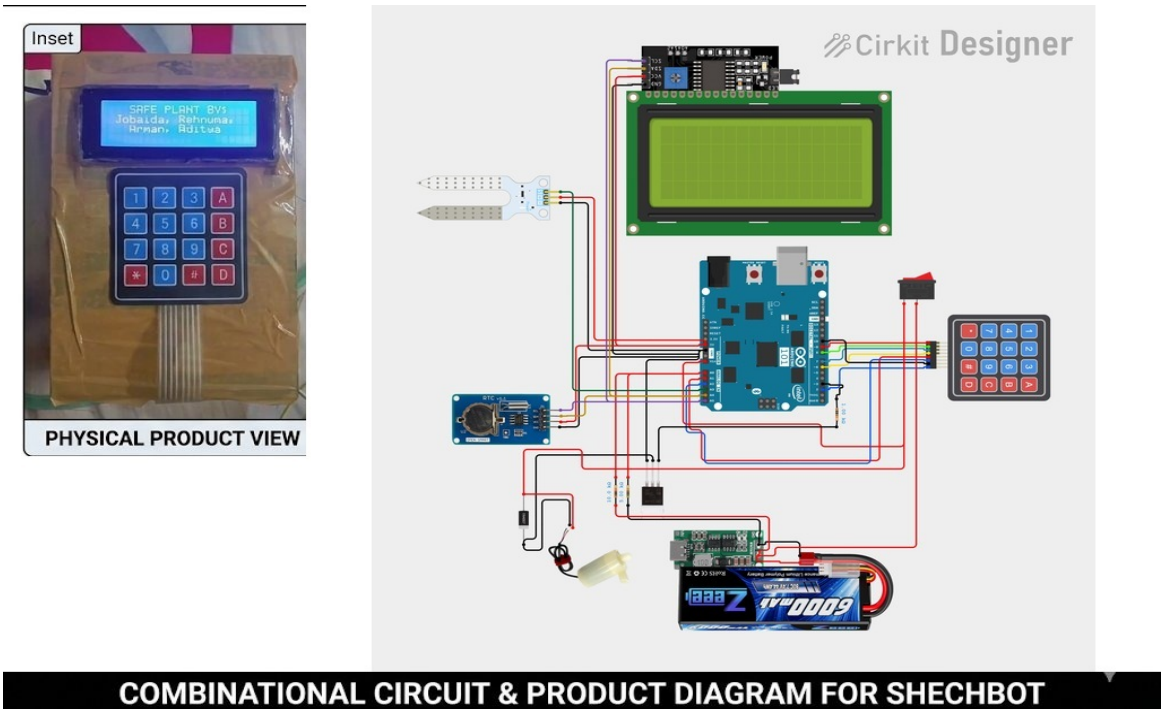

Figure 1 — Combinational circuit and product diagram for ShechBot.

3.2 Firmware Architecture — State Machine Design

The firmware is structured as a finite-state machine (FSM). On startup, settings load from EEPROM (falling back to safe defaults on checksum failure). Eight states govern all system behaviour:

State	Description
MAIN_MENU	Root navigation hub — routes to all other states via keypad.
MODE_TIME	Time-based monitoring — polls DS3231 and triggers pump at the scheduled time.
MODE_MOIST	Moisture monitoring — continuously compares live sensor value to threshold.
SET_MOIST	Configuration state for adjusting the target moisture percentage.
SET_HOUR	Schedule configuration step 1 — accepts hour input (0–23).
SET_MIN	Schedule configuration step 2 — accepts minute input (0–59).
SET_PUMP	Configuration state for pump run duration in seconds (1–300).
VIEW_BATT	Displays real-time battery voltage and percentage charge estimate.

Table 2 — FSM state definitions.

3.3 Keypad Navigation Reference

Key	Action
[A]	Enter Time-Based Monitoring Mode
[B]	Enter Moisture-Based Monitoring Mode
[C]	Set Scheduled Watering Time (Hour → Minute)
[D]	Set Target Soil Moisture Threshold (%)
[0]	Set Pump Run Duration (seconds)
[*]	View Live Battery Status
[#]	Return to Main Menu / Confirm / Exit current screen

Table 3 — Keypad function mapping.

4. Code Implementation

4.1 Libraries and Hardware Initialisation

Standard Arduino libraries included at sketch top. LCD initialises over I2C; RTC uses the **RTClib** wrapper; keypad declared with 4×4 character map:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>    #include <Keypad.h>
#include <RTClib.h>                #include <EEPROM.h>

LiquidCrystal_I2C lcd(0x27,20,4);    RTC_DS3231 rtc;
char keys[4][4]={{'1','2','3','A'},{'4','5','6','B'},
                 {'7','8','9','C'},{'*','0','#','D'}};
byte rowPins[4]={9,8,7,6};    byte colPins[4]={A1,A2,10,2};
Keypad kp=Keypad(makeKeymap(keys),rowPins,colPins,4,4);
```

■ *Initialises LCD (I2C), DS3231 RTC, and 4×4 keypad for interaction and timekeeping.*

4.2 EEPROM-Based Persistent Settings

All configurable parameters stored as a **Settings** struct in EEPROM. A 16-bit XOR checksum validates data on every boot; mismatches silently restore safe defaults (50 % moisture, 16:00 schedule, 30 s pump duration):

```
struct Settings {
    uint16_t moisture, hour, minute, pumpSeconds, checksum;
};
void loadSettings() {
    EEPROM.get(0, settings);
    if (computeChecksum(settings) != settings.checksum)
        { settings = DEFAULT_SETTINGS;  saveSettings(); }
}
```

■ *Stores parameters persistently with XOR checksum validation — survives power outages.*

4.3 Soil Moisture Measurement and Pump Control

Raw ADC values (1023 = dry, 200 = wet) mapped to 0–100 %. IRFZ44N MOSFET drives pump on PWM pin D3 with smooth ramp-up; 1N4007 diode suppresses back-EMF on shutdown:

```
int getMoisture() {
    return constrain(map(analogRead(sensorPin),1023,200,0,100),0,100);
}
void triggerPump() {
    pumpRunning=true;  pumpStartTime=millis();
    for(int i=0;i<=255;i+=51){analogWrite(pumpPin,i);delay(40);}
    lcd.clear();  lcd.setCursor(4,1);  lcd.print("PUMP ACTIVE");
}
```

■ *Converts ADC reading to percentage and activates pump with PWM ramp for surge protection.*

4.4 Time-Based Irrigation Logic

DS3231 RTC polled every second. When current time matches the configured schedule and day not yet processed, soil moisture is checked — pump fires if below threshold, otherwise a 'Moisture OK' message displays:

```
void checkTimeIrrigation() {
    DateTime now = rtc.now();
    if (now.hour()==settings.hour && now.minute()==settings.minute
        && now.day()!=lastWateredDay) {
        lastWateredDay = now.day();
        if (getMoisture()<settings.moisture) triggerPump();
        else  timeMessageActive = true;
    }
}
```

■ *Triggers once per calendar day at the configured time, only if soil is dry enough.*

4.5 Moisture-Based Irrigation Logic

Sensor sampled every 500 ms. A 5-minute cooldown (COOLDOWN = 300000UL) prevents repeated triggering before water diffuses through the soil. Pump activates reactively whenever moisture drops below the target threshold:

```
void checkMoistIrrigation() {
    if (millis()-lastMoistureWatering < COOLDOWN) return;
    if (getMoisture() < settings.moisture) {
        triggerPump();  lastMoistureWatering = millis();
    }
}
```

4.6 Battery Voltage Monitoring

A 10 kΩ / 2.2 kΩ voltage divider scales the 7.4 V pack to 5 V ADC range. Sample reconverted and linearly mapped to 0–100 % charge estimate:

```
const float V_REF=5.0, R1=10000.0, R2=2200.0;
const float BATT_MAX=12.6, BATT_MIN=10.5;
void runBatteryDisplay() {
    float v=(analogRead(battPin)*V_REF/1024.0)*((R1+R2)/R2);
    int pct=constrain(map(v*10,BATT_MIN*10,BATT_MAX*10,0,100),0,100);
    lcd.print(v,2);  lcd.print(" V  ");  lcd.print(pct);  lcd.print("%");
}
```

■ *Provides real-time voltage and charge percentage — updated every second.*

4.7 Main Loop — State Machine Overview

The **loop()** function handles pump timeout, 60 s screen-saver backlight cutoff, and routes keypad input to the active state:

```
void loop() {
    if (pumpRunning &&
        millis()-pumpStartTime>=settings.pumpSeconds*1000UL) stopPump();
    char key=kp.getKey();
    if (key) lastInteractionTime=millis();
    if (millis()-lastInteractionTime>60000) lcd.noBacklight();
    switch(currentState) {
        case MODE_TIME:  runTimeModeDisplay(); checkTimeIrrigation(); break;
        case MODE_MOIST: runMoistModeDisplay();checkMoistIrrigation();break;
        case VIEW_BATT:  runBatteryDisplay();                          break;
    }
}
```

■ *Central FSM dispatcher — pump timing, screen-saver, keypad input, and mode routing.*

5. Experimental Setup and Testing

The system was evaluated over a **30-day** continuous field trial. The following metrics were recorded to assess timing accuracy, sensor reliability, and power performance:

Metric	Result	Observations
RTC Time Drift	< 3 sec / month	DS3231 TCXO ensures accurate long-term scheduling
Scheduled Events	96.7 % success	One failure attributed to intermittent wiring contact
Moisture Override	3 / 3 correct	All sensor-triggered events activated correctly
Screen-Saver Trigger	100 % reliable	Backlight cut at 60 s inactivity in all tests
EEPROM Persistence	Pass (all cycles)	Settings survived 12 simulated power interruptions
Battery Monitoring	±0.05 V	Voltage divider calibrated against known reference

Table 4 — 30-day experimental validation results.

6. Known Limitations and Flaws

Two hardware limitations were identified during development and field testing. Both are addressable in a revised PCB iteration.

#	Issue	Description	Recommended Fix
1	Loose Wiring Connections	Breadboard and jump-wire contacts develop intermittent resistance under pump vibration, causing LCD glitches, erratic sensor readings, or pump activation failures.	Solder all connections to a custom PCB; add strain relief to all wiring harnesses.
2	RTC Coin-Cell Depletion	The DS3231 uses a CR2032 coin cell to hold time when the main battery is disconnected. As it drains over months, incorrect time causes scheduling errors.	Monitor VBAT pin or temperature drift; display low-battery warning on LCD and prompt user to re-sync the time.

Table 5 — Identified hardware limitations and remediation strategies.

7. Conclusion

ShechBot demonstrates that a capable, standalone smart-irrigation controller can be realised from commodity Arduino hardware at minimal cost. The combination of DS3231-based precise scheduling, real-time soil moisture sensing, EEPROM settings persistence, and an intuitive LCD/keypad interface delivers a robust device suitable for both domestic and small-scale agricultural deployment. The two identified limitations — loose wiring and RTC coin-cell depletion — are correctable in a next-revision PCB design. Future extensions may incorporate multi-zone pump control, SD-card data logging, or wireless reporting via ESP8266 for remote monitoring without sacrificing off-grid capability.

Summary of Key Achievements

- Dual-mode irrigation (time-scheduled + moisture-triggered) fully operational.
- EEPROM persistence with XOR checksum — settings survive all power interruptions.
- Real-time battery monitoring via voltage divider — accurate to ±0.05 V.
- Interactive menu — all parameters adjustable in the field without re-programming.
- 60-second LCD screen-saver extends battery life during unattended operation.

Full Source Code — ShechBot

https://tinyurl.com/ShechBot-Code

Scan the QR code or visit the URL above to access the complete annotated Arduino sketch (.ino) hosted on GitHub Gist.